

Open-Source Code Refactoring

1. INTRODUCTION

Code refactoring improves nonfunctional attributes of the software without altering the behavior. The purpose of this assignment is to identify code smells in a real-world, open-source Python repository, establish a testing safety net with `pytest`, and perform distinct refactorings to improve code maintainability and cyclomatic complexity.

2. REPOSITORY SELECTION

- **Original Project name:** Battleship
- **Original repository URL:** <https://github.com/gbrough/battleship>
- **My refactored repository URL:** <https://github.com/mitanshm18/Software-engineering-Lab-6>
- **Programming language:** Python
- **Initial LOC:** 155 lines of code for the main `battleship.py` application.
- **Project description:** A CLI-based Battleship game featuring a human player versus an automated computer opponent.

3. CODE SMELL AUDIT

#	Code Smell	File	Function	Explanation	Refactoring
1	Duplicate Code	<code>battleship.py</code>	<code>place_ships</code>	The exact same <code>for</code> loops were used to place ships for the player and computer depending on orientation.	Extract Method
2	Magic Numbers	<code>battleship.py</code>	<code>check_ship_fit</code>	Hardcoded <code>8</code> was used multiple times to check if ships exceeded the grid.	Replace Magic Numbers with Constants
3	Complex Conditional	<code>battleship.py</code>	<code>user_input</code>	Huge nested <code>while True</code> and <code>try...except</code> blocks checking input validity. CCN was 18.	Simplify Conditional (Extract helper)
4	Poor Naming	<code>battleship.py</code>	<code>count_hit_ships</code>	A variable iterating over elements in a row was named <code>column</code> , making it sound like an integer index rather than a cell object.	Rename Variable

4. TESTING STRATEGY & SAFETY NET

- **Existing test suite:** None
- **Testing framework:** `pytest` and `unittest.mock.patch`
- **Coverage command:** `python -m pytest --cov=battleship test_battleship.py`
- **Final coverage:** 78% Statement Coverage
- **Tests Execution:** Passed successfully before and after refactoring.

5. SYSTEMATIC REFACTORING OPERATIONS

1. **Extract Method:** Abstracted the duplicated matrix modification logic into a dedicated `place_ship_on_board(board, row, column, orientation, ship_length)` function.
2. **Replace Magic Numbers:** Defined `BOARD_SIZE = 8` globally and dynamically sized arrays and condition bounds (e.g., `row + SHIP_LENGTH > BOARD_SIZE`).
3. **Simplify Conditional:** Created a small generic `get_valid_input(prompt, valid_options)` helper to collapse the three massive while-loops inside `user_input`.
4. **Rename Variable:** Renamed the confusing `column` to `cell`.

6. BEFORE VS AFTER METRICS

Metric	Before	After	Change
Average Cyclomatic Complexity	8.0	5.3	Improved (-33.7%)
<code>user_input</code> function Complexity	18	4	Improved (-77.7%)
Lines of Code (NLOC)	155	138	Improved (-11%)

Explanation: Extracting the input validation logic drastically reduced the complexity footprint of `user_input`. Removing duplicate code in `place_ships` physically shrank the codebase size.

7. GIT COMMIT HISTORY

Commit Message

```
Initial import of original python Battleship project
chore: rename main file to battleship.py for easier import
test: add test suite with 76% coverage
refactor: extract place_ship_on_board method to remove duplicate code
refactor: replace magic numbers with BOARD_SIZE constant
refactor: simplify conditionals and loops with get_valid_input helper
refactor: rename confusing 'column' variable to 'cell'
```

8. REFLECTION

This project demonstrated the immense power of `pytest` and `unittest.mock`. Because the CLI game heavily relied on `input()` and randomness, it was initially untestable. By heavily

mocking `input` via `@patch`, we achieved strong branch coverage, which made dropping the CCN of `user_input` from 18 to 4 completely risk-free.